



DESIGN BIBLE

DEATHTRAP DUNGEON

 Aurélio A. Júnior

 Gabriel G. Figueiredo



Introdução — Motivação



- Projeto Unity/C#
- Traduzir decisões de design em código funcional
- Base prática para a disciplina de Desenvolvimento de Jogos

Introdução — Objetivos



- Consolidar o Design Bible do projeto Deathtrap Dungeon
- Mapear pilares de design, loop de gameplay e arquitetura técnica
- Avaliar o escopo implementado frente ao planejado

Design Bible — Roteiro

- Aventureiro x masmorra hostil x chefe final
- Combate ágil, progressão incremental e exploração em Tilemap
- Gênero: Action RPG / Hack-and-slash top-down (Roguelike)
- Plataforma: PC (Unity) · Câmera top-down 2D

Estrutura de Fases

- Begin → Dungeon_0 → Dungeon_1 → Dungeon_Boss → End
- Transições por Portais, com salvamento automático de progresso

Game Design — Pilares



- Combate tátil e com peso
- Risco recompensado por fúria
- Progressão legível (sem aleatoriedade)
- Mundo legível por meio do Tilemap

Progressão do Jogador



- XP → nível → vida máxima (+10 por nível)
- Ouro → nível da arma → mais dano e empurrão
- Bestiário: 8 inimigos comuns + 1 chefe (Boss0, 150 HP)

Game Play — Loop Central



- Explorar → Combater → Ganhar XP/Ouro → Evoluir → Avançar → Chefe

Sistema de Combate



- Ataque por clique/espaco, cooldown de 0,4s
- Animação em 4 fases (preparação, corte, finalização, recolhimento)

Sistema de Fúria (Rage)



- Barra de 0 a 50 · dano recebido vira Fúria
- Ativa a "Espada Flamejante" por 4 segundos

Interface Gráfica — UI/HUD

- HUD em tempo real: vida, XP e fúria
- Textos flutuantes: dano, cura, XP, "LEVEL UP!"
- Tela de transição de cena com barra de progresso

Arte, Estilo e Áudio

- Pixel art top-down, tileset 16×16, masmorra sombria
- Sinalização por cor: chefe fica vermelho em enrage
- BGM em loop + efeitos sonoros dinâmicos por evento

Arquitetura e Padrões de Projeto

- Hierarquia: Fighter → Mover → Player / Enemy
- Singleton para estado global (ouro, XP, save)
- Dano via mensageria (SendMessage)
- Fases de ataque implementadas no Animator

Save/Load & Limitações

- Persistência em JSON, salva automaticamente nos Portais
- Nível recalculado a partir do XP total salvo
- Limitação em aberto: knockback atravessando paredes

Metodologia de Avaliação



- Engenharia reversa documental do código-fonte
- Cruzamento com planilha de balanceamento e README
- Validação entre design pretendido e código implementado

Resultados e Discussões



- Sistemas centrais funcionais: combate, progressão, save/load
- Balanceamento: custo de arma acelerado x XP linear
- Bugs de respawn corrigidos; knockback em paredes em aberto
- Slot de inimigo (enemy_10) planejado, ainda não implementado

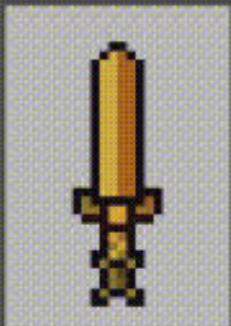
Conclusões

- Pilares de design refletidos de forma consistente no código
- Base funcional e coerente, pronta para expansão
- Estudo de caso adequado à disciplina de Desenvolvimento de Jogos

Trabalhos Futuros

- IA de inimigos em repouso · combate entre inimigos
- Correção do knockback atravessando paredes
- Expansão do bestiário e otimização adicional de memória




MAX


< >

HEALTH
15 / 20
LEVEL
2
PESOS
360

0 / 30

Game Tile Palette Animation Timeline
Display 1 800x600 Scale 0.685 Maximize On Play Mute Audio VSync Stats Gizmos

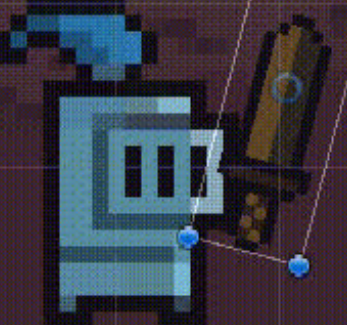

HEALTH
10 / 10
LEVEL
1
PESOS
500


10


< >

8 / 10





Tilemap
Focus On None

Game Tile Palette Animation Timeline
Preview weapon_swing Samples 60
weapon : Position
weapon : Rotation
weapon : Box Collider 2D.Enabled



Referências



Fontes consultadas para o Design Bible

- Repositório de referência do projeto (LOUMERIM/DeathtrapDungeon)
- Ferramentas de Tilemap — Unity Technologies (2d-extras)
- Biblioteca de serialização JSON — LitJson
- Código-fonte C# e planilha de dados de balanceamento (GameData)

